

EIMS Documentation

Welcome to the official Master Project Documentation for the BigOutsource Employee Information Management System (EIMS). This document serves as a comprehensive, single-source-of-truth manual for software engineers, DevOps personnel, IT administrators, and non-technical management staff.

1. Project Overview & Tech Stack

Business Purpose & High-Level Goals

The BigOutsource Employee Information Management System (EIMS) is an enterprise workforce and asset management portal tailored specifically for BigOutsource operations. The application centralizes:

- **Employee Lifecycle & Directory Management:** Full tracking of employee identity, HR details, employment status, evaluation milestones, emergency contacts, and government IDs.
- **IT Credentials & Security Provisioning:** Dedicated management of corporate emails, LMS accounts, workstation assignments, REMOTE ID tools, antivirus (ESET) and activity monitoring (ActivityWatch) statuses, and disk encryption keys.
- **Hardware Asset Management:** Detailed inventory tracking of company assets (laptops, desktops, monitors, peripherals), license keys, assignment histories, and device status updates.
- **DEPARTMENT/CAMPAIGN & Site Organization:** Categorization of workforce units across multiple physical sites (HQ - San Pablo, Candelaria, WFH, Hybrid) and dynamic department/campaign assignments.
- **Bulk Data Staging & Validation:** Staging, duplicate detection, and import workflows for large-scale employee onboarding via Excel/CSV files.
- **Automated Alerts & Scheduling:** Real-time notifications and automated email digests for daily birthdays and 1st, 3rd, 5th, 6th month, and anniversary employee evaluation milestones.
- **Granular Role-Based Access Control (RBAC):** Fine-grained security model enforcing capability-based authorization across system views, editing privileges, export rights, and sensitive secret visibility.

Tech Stack Breakdown

Frontend Layer

Component	Technology	Version	Purpose
Framework	React	^19.0.1	UI render engine with functional component architecture
Language	TypeScript	~5.8.2	Type safety, interfaces, and compile-time contract enforcement
Build Tool	Vite	^6.2.3	Lightning-fast HMR dev server and production bundling
Styling	Tailwind CSS	^4.1.14	Utility-first responsive styling engine with @tailwindcss/vite
Routing	React Router DOM	^7.15.1	Client-side routing, protected routes, and dynamic sub-paths
Data Fetching	TanStack React Query	^5.101.0	Asynchronous state management, caching, and cache invalidation

Data Tables	TanStack React Table	^8.21.3	Headless table state logic for sorting, filtering, and pagination
Realtime Client	Socket.io Client	^4.8.3	Client WebSocket connection for real-time presence and alerts
UI Components & Icons	Lucide React / Framer Motion	^0.546.0 / ^12.39.0	Modern UI icon library and smooth layout animation primitives
Export Utilities	ExcelJS & XLSX	^4.4.0 / ^0.18.5	Client-side Excel spreadsheet parsing and custom workbook exports

Backend Layer

Component	Technology	Version	Purpose
Runtime Engine	Node.js (ES Modules)	>=18.0.0	Asynchronous I/O JavaScript server runtime (type: module)
API Framework	Express.js	^4.22.2	RESTful HTTP router and middleware pipeline

ORM / Query Engine	Prisma Client & CLI	^6.19.3	Type-safe PostgreSQL schema management and database access
Realtime Engine	Socket.io Server	^4.8.3	Server WebSocket hub for active user presence and instant alerts
Security & Auth	JWT & BcryptJS	^9.0.3 / ^3.0.3	Stateless Bearer token creation and salted password hashing
Rate Limiting	Express-Rate-Limit	^7.5.1	IP-based request throttling to prevent denial of service / brute force
HTTP Security	Helmet & CORS	^8.1.0 / ^2.8.6	HTTP response headers hardening and domain origin restriction
File Handling	Multer	^2.2.0	Multipart form-data handling for employee avatar image uploads
Task Scheduler	Node-Cron	^4.6.0	Scheduled background cron jobs for daily evaluation and birthday checks

Email Transporter	Nodemailer	^9.0.3	SMTP transport integration for MFA OTPs and system notifications
-------------------	------------	--------	--

Database & Storage Layer

Component	Technology	Version	Details
Database Engine	PostgreSQL	15-alpine	Primary relational database running in Docker container (eims)
File Storage	Local Disk (/uploads)	Disk Storage	Server-side directory storing user and employee avatar uploads

Infrastructure & External Tools

Component	Technology	Details
Containerization	Docker & Docker Compose	Container orchestrator for backend, frontend (Nginx), and database
Local SMTP Interceptor	Mailpit	Docker service (Port 8025/1025) capturing outbound emails during local development

Reverse Proxy	Nginx	High-performance HTTP web server serving Vite builds and proxying /api requests
---------------	-------	---

2. Local Environment Setup & Directory Structure

Prerequisites & Tools

Ensure the following tools are installed on your workstation prior to setting up the project:

- Node.js: v18.20.0 or v20.x LTS recommended
- npm: v9.x or v10.x
- Docker Desktop (or Docker Engine + Docker Compose v2+): Required for containerized database and full-stack local deployment
- Git: v2.x+

Step-by-Step Local Setup Instructions

Step 1: Clone the Repository

```
git clone https://github.com/Makixenn/bigoutsource-eims.git
cd bigoutsource-eims
```

Step 2: Install Node Dependencies

Install dependencies for both backend and frontend applications:

```
# Install backend dependencies
cd backend
npm install

# Install frontend dependencies
cd ../frontend
npm install
cd ..
```

Step 3: Configure Environment Variables

Create the .env files in both the backend and frontend subdirectories.

Backend Environment File (backend/.env):

```
NODE_ENV=development
PORT=5001
CORS_ORIGIN=http://localhost:3000,http://localhost:3001,http://localhost:3002
DATABASE_URL=postgresql://postgres:postgres@127.0.0.1:5433/eims?schema=public
JWT_SECRET=super_secret_jwt_key_for_local_dev

# Seed Super Admin credentials (automatically created on server boot)
SEED_SUPER_ADMIN_EMAIL=kamote@gmail.com
SEED_SUPER_ADMIN_PASSWORD=kamote123
SEED_SUPER_ADMIN_FULL_NAME=Local Super Admin
SEED_SUPER_ADMIN_DEPARTMENT=Administration
SEED_SUPER_ADMIN_SITE=HQ

# SMTP Configuration (Uses Mailpit in Docker or real SMTP server)
SMTP_HOST=localhost
SMTP_PORT=1025
SMTP_USER=
```

1. SMTP_PASS=
2. Frontend Environment File (frontend/.env):
VITE_API_BASE_URL=http://localhost:5001/api

Step 4: Launch Database via Docker Compose

Start the PostgreSQL database (and optionally Mailpit) container:

```
docker compose up -d db
```

Note: Docker maps PostgreSQL host port 5433 to container port 5432 to prevent conflicts with existing local PostgreSQL installations.

Step 5: Initialize Database Schema & Seed Data

Navigate to the backend directory and push the Prisma schema to the PostgreSQL database:

```
cd backend
npx prisma db push
```

```
npx prisma generate
cd ..
```

Step 6: Start Local Development Servers

You can run the backend and frontend development servers concurrently in two separate terminal windows:

Terminal 1 (Backend API):

```
cd backend
```

- `npm run dev`
(Starts Node server with `--watch` on `http://localhost:5001`, bootstraps the Super Admin, initializes WebSockets, and starts background schedulers).

Terminal 2 (Frontend Web App):

```
cd frontend
```

- `npm run dev`
(Starts Vite dev server on `http://localhost:3000`).

Access the application in your browser at `http://localhost:3000` and log in with `kamote@gmail.com / kamote123`.

Repository Directory Map

```
bigoutsource-eims/
├── .agents/                # Workspace-specific agent configurations & skills
│   └── skills/             # Custom operational skills (e.g. Supabase, Postgres)
├── backend/                # Node.js + Express API server
│   ├── prisma/
│   │   └── schema.prisma   # Prisma ORM database models & migrations definition
│   ├── src/
│   │   ├── config/        # Environment configs, DB client, capabilities matrix
│   │   │   ├── capabilities.js # Master catalog of RBAC permissions and default roles
│   │   │   ├── db.js       # Prisma client instantiation
│   │   │   └── env.js       # Centralized environment variable loader
│   │   ├── controllers/    # Request handlers (Auth, Employee, Assets, AuditLogs,
│   │   │   etc.)
│   │   └── middleware/     # Auth, permission, rate limiting, validation & error
├── handlers
│   ├── models/             # Data access abstraction models
│   └── realtime/           # Socket.io server initialization & presence tracking
```

		routes/	# Express API route endpoints definitions
		services/	# Core business logic, email dispatch, cron schedulers
		utils/	# Validators, upload handling, security helpers
		uploads/	# Disk directory for stored employee profile avatars
		Dockerfile	# Production Docker build file for Node API
		Dockerfile.dev	# Development Docker container file
		package.json	# Server dependencies and lifecycle scripts
		frontend/	# React 19 + TypeScript + Vite web application
		public/	# Static brand assets (favicons, logos)
		src/	
		components/	# Shared UI components (Navigation, Modals, Tables,
		Alerts)	
		contexts/	# Global React contexts (Auth, Theme, TextSize,
		Presence)	
		features/	# Modular domain features (Auth, Dashboard, Employees,
		Reports)	
		hooks/	# Custom React hooks (debounce, responsive, queries)
		lib/	# Permission helpers (`userCan`), API client instances
		pages/	# Primary route view components (Directory, Profile,
		Assets, etc.)	
		providers/	# TanStack Query & Application providers
		App.tsx	# Main routing component with ProtectedRoute guards
		index.css	# Modern Tailwind CSS configuration & root variables
		main.tsx	# Web app mounting entry point
		nginx.conf	# Nginx server configuration for containerized
		deployment	
		Dockerfile	# Multi-stage Dockerfile compiling React to static
		Nginx build	
		package.json	# Frontend dependencies and build scripts
		vite.config.ts	# Vite build options, path aliases, proxy rules
		docker-compose.yml	# Container configuration for development
		DOCUMENTATION.md	# Master Project Documentation file

Common Setup Troubleshooting Steps

1. Port 5001 or 3000 Already in Use:

- *Symptom:* Server throws EADDRINUSE: address already in use :::5001.
- *Solution:* Identify process running on that port using `netstat -ano | findstr :5001` (Windows) or `lsof -i :5001` (Linux/Mac) and terminate it, or adjust PORT in `backend/.env`.

2. Database Connection Refused:

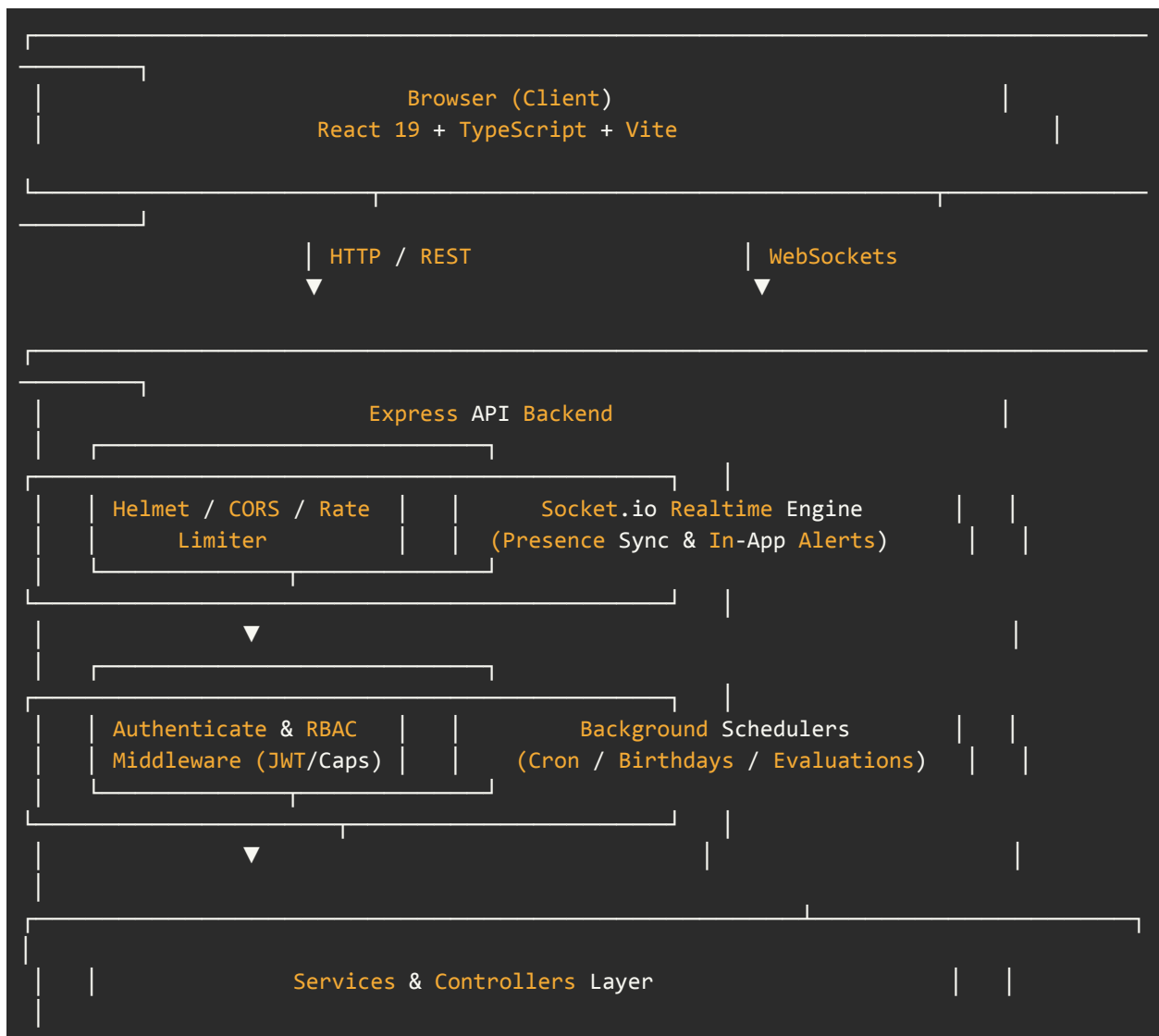
- *Symptom:* PrismaClientInitializationError: Can't reach database server at 127.0.0.1:5433.
- *Solution:* Ensure Docker container is up (`docker compose ps`). Check if the database port mapping in `docker-compose.yml` (5433:5432) matches the DATABASE_URL port in `backend/.env`.

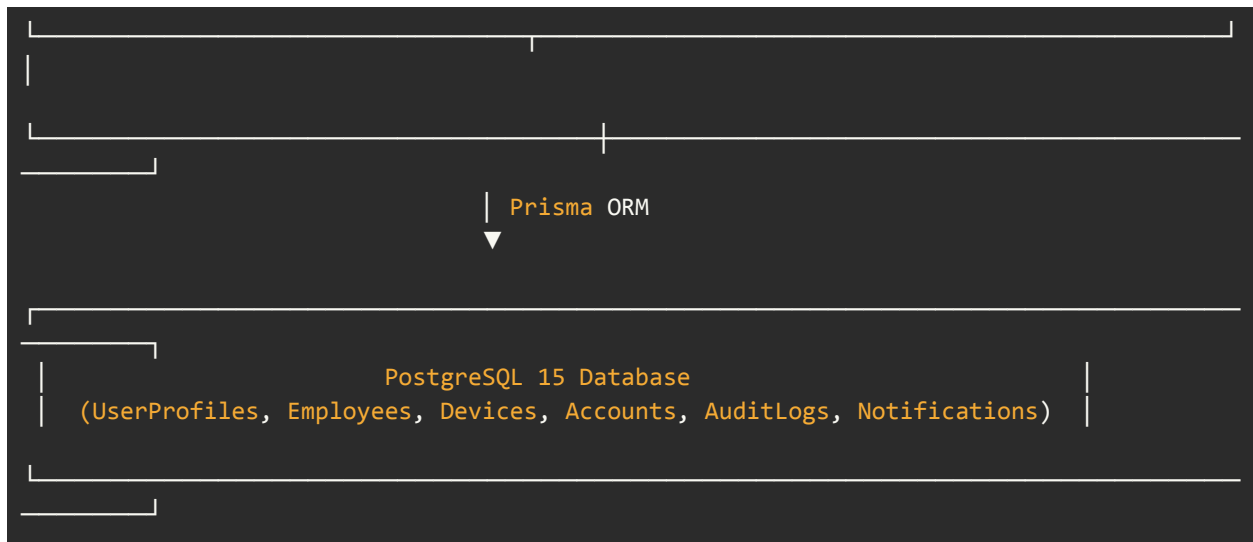
3. CORS Network Error on API Requests:

- *Symptom*: Browser console blocks requests with CORS error.
 - *Solution*: Verify CORS_ORIGIN in backend/.env contains your exact frontend origin (e.g., http://localhost:3000).
4. Missing Prisma Client Generator:
- *Symptom*: Cannot find module '@prisma/client'.
 - *Solution*: Run npx prisma generate inside the backend directory.

3. Architecture & Security

System Architecture & Data Flow Overview





Data Flow Sequence:

1. User interacts with client UI pages. React Query dispatches signed HTTP requests containing Authorization: Bearer <JWT> to Express /api/*.
2. Helmet checks security headers, CORS origin validator verifies client domain, and rate-limiting middleware evaluates request counts.
3. authenticate middleware decodes JWT, confirms active user profile in database, and enforces single-session validity via currentSessionId.
4. requirePermission or requireAnyPermission middleware checks effective capabilities against the endpoint requirement.
5. Controller processes business logic via Service/Model classes using Prisma ORM.
6. Audit logs are logged automatically for mutating actions.
7. Socket.io broadcasts realtime presence changes or notification payloads to connected clients.

Database Schema Summary / Primary Data Models

The database contains 8 core models managed in backend/prisma/schema.prisma:

1. UserProfile (user_profiles):
 - Stores system administrative user accounts.
 - Fields: id, email (unique), passwordHash, fullName, role, status (pending, active, disabled), department, site, capabilityOverrides (String array), approvedById, approvedAt, passwordSetupToken (unique), resetPasswordToken (unique), resetPasswordExpires, currentSessionId.
2. Employee (employees):

- Central repository of workforce members.
 - Fields: id, name, account (DEPARTMENT/CAMPAIGN assignment), phoneNumber, address, bigoutsourceEmail, emailPassword, lmsAccount, status (active, pending, separated, floating), employeeStatus (Regular, Probationary, Contractual), site, pcName, deviceType, biosDate, windowsLicenseKey, rustdeskId (REMOTE ID), eset, activitywatch, diskEncryptionKey, dateHired, separationDate, separationReason, macAddresses (JSON array), isArchived, nickname, position, sex, civilStatus, sssNo, tinNo, philhealthNo, pagibigNo, personalEmail, mainContact, emergencyContact, emergencyContactNumber, birthdate, floatDate, isReadyForArchive, provisioningStatus, archiveInitiator, archiveItClearance, archiveHrClearance, archiveOpsClearance, archiveFinanceClearance, outlookEmail, mattermostAccount, teamsAccount, avatarUrl, idIssuance, hoodieIssuance, hmoEnrollment, hmoMemberCode, evalFirstMonth, evalThirdMonth, evalFifthMonth, evalSixthMonth, evalAnniversary.
3. Account (accounts):
 - Master reference list for DEPARTMENT/CAMPAIGN units within BigOutsource.
 - Fields: id, name (unique), accountType (internal, external), departmentCode, lastUsedAt.
 4. Role (roles):
 - Configurable access control roles.
 - Fields: slug (Primary Key), name, isSystem (Boolean), capabilities (String array of permission keys).
 5. AuditLog (audit_logs):
 - Full immutable record of system operations for compliance auditing.
 - Fields: id, userId, userEmail, userName, userRole, action, entityType, entityId, entityLabel, details (JSON), ipAddress, userAgent, createdAt.
 6. Notification (notifications):
 - System and user notifications.
 - Fields: id, recipientId, type, actorId, actorName, actorRole, message, entityType, entityId, entityLabel, actionUrl, details (JSON), readAt, createdAt.
 7. EmployeeImportStaging (employee_import_staging):
 - Temporary staging area for bulk spreadsheet imports prior to finalizing database records.
 - Fields: id, importBatchId, sourceSheet, sourceRow, rawData (JSON), normalizedData (JSON), issues (JSON), status, duplicateKey, resolution (JSON), createdById.
 8. AppSettings (app_settings):
 - System-wide administrative flags.
 - Fields: id (default "global"), companyName, notifyRegistrationAttempts, notifySystemAlerts.

Authentication & Authorization Implementation

Authentication Flow & MFA

- Authentication Standard: JSON Web Token (JWT) with standard Authorization: Bearer <token> HTTP header.
- Mandatory Email OTP (MFA):
 - Phase 1: User submits email & password to /api/auth/login. System validates credentials and generates a 6-digit OTP sent via Nodemailer to the user's email. A temporary mfaToken (valid 5 minutes) is returned.
 - Phase 2: User submits 6-digit OTP to /api/auth/login/mfa. Upon verification, the backend issues full access JWT payload containing sessionId.
- Single-Session Enforcement: When a user logs in, a unique sessionId UUID is saved to UserProfile.currentSessionId. Subsequent API requests verify that token.sessionId === profile.currentSessionId. A new login immediately invalidates previous sessions across all devices.
- Link-Based Account Password Setup: Admin-created user accounts generate a secure passwordSetupToken sent via email. Clicking the link allows the employee to set their initial password and complete MFA without requiring admins to communicate plain-text temporary credentials.

Role-Based Access Control (RBAC) & Capability Matrix

Authorization in EIMS is capability-driven rather than role-string hardcoded. 60+ granular capabilities control UI views, endpoint access, and field-level visibility:

- Seeded Default Roles:
 1. super_admin: Has all capabilities (including users.manage, roles.manage, settings.manage).
 2. director: Full read/write across HR, IT, Assets, Reports, Archiving, and Audit Logs.
 3. hr_admin: Focused on employee HR fields, evaluation schedules, department rosters, HR exports, and daily birthday alerts.
 4. it_admin: Focused on IT provisioning, remote access IDs, software keys, computer asset inventory, and IT exports.
 5. viewer: Read-only access to basic employee directory and departments.

Security Measures

- **CORS Policy:** Configured in backend/src/app.js using origin verification. Development allows localhost ports (http://localhost:3000, 3001, 3002); production restricts access strictly to domains specified in CORS_ORIGIN.
- **Rate Limiting:**
 - Global API limiter: 5000 requests per 15-minute window for /api/* endpoints.
 - Login & Auth limiter: 100 requests per 15-minute window for sensitive auth routes.
- **HTTP Hardening:** Managed via helmet:
 - Enforces frameguard: { action: 'deny' } to protect against Clickjacking.
 - Content Security Policy (CSP) restricts allowed scripts, styles, and connect origins.
 - Prevents API response caching (Cache-Control: no-cache, no-store, must-revalidate).
- **Input Validation & Sanitization:** Express middleware layer uses express-validator across auth, employee, device, and department routes to enforce email format, string sanitization, and password strength rules (min 12 chars, uppercase, lowercase, number, special char).

4. API Reference

Base URL & Request Headers

- Local Dev Base URL: http://localhost:5001/api
- Production Base URL: https://api.yourcompany.com/api

Default Headers:

Content-Type: application/json

Authorization: Bearer <JWT_ACCESS_TOKEN>

Master API Endpoints Table

Category	HTTP Method	Route	Description	Auth / Required Capability

Auth	POST	/api/auth/register	Register new user account	Public
Auth	POST	/api/auth/login	Step 1 login (authenticates & dispatches MFA OTP)	Public (Rate-Limited)
Auth	POST	/api/auth/login/mfa	Step 2 login (verifies OTP & returns session JWT)	Public (Rate-Limited)
Auth	GET	/api/auth/me	Retrieve current authenticated user profile & capabilities	Authenticated
Auth	PUT	/api/auth/me	Update current user profile details	Authenticated
Auth	POST	/api/auth/logout	Revoke current user session	Authenticated
Auth	POST	/api/auth/setup-password	Complete initial password setup via token	Public

Employee s	GET	/api/employees	List employee directory with filtering/pagination	employees.view
Employee s	GET	/api/employees/summary	Get employee count & status analytics	employees.view
Employee s	GET	/api/employees/:id	Fetch single employee record	employees.view
Employee s	POST	/api/employees	Create new employee record	employees.create
Employee s	PUT	/api/employees/:id	Update employee profile fields	employees.edit / employees.it.edit
Employee s	DELETE	/api/employees/:id	Archive or permanently delete employee	archiving.finalize / archiving.initiate
Employee s	POST	/api/employees/:id/avatar	Upload employee avatar photo	employees.edit / employees.it.edit
Employee s	POST	/api/employees/:id/remind- it	Send IT provisioning reminder email	employees.edit

Imports	GET	/api/employee-imports	List staging import rows	imports.manage
Imports	POST	/api/employee-imports/stage	Stage raw CSV/Excel rows for import processing	imports.manage
Imports	POST	/api/employee-imports/:batchId/import-ready	Commit staged valid rows to employee database	imports.manage
Assets	GET	/api/devices	List hardware asset inventory	assets.view
Assets	POST	/api/devices	Add new hardware asset	assets.edit
Assets	PUT	/api/devices/:id	Edit hardware asset details	assets.edit
Assets	DELETE	/api/devices/:id	Delete hardware asset	assets.edit
Assets	POST	/api/device-assignments	Assign hardware asset to employee	assets.edit

Assets	PUT	/api/device-assignments/:id /return	Mark hardware asset assignment as returned	assets.edit
Departments	GET	/api/accounts	List all DEPARTMENT/C AMPAIGN units	departments.view
Departments	POST	/api/accounts	Create new DEPARTMENT/C AMPAIGN unit	departments.edit
Departments	PUT	/api/accounts/:id	Update DEPARTMENT/C AMPAIGN unit	departments.edit
Departments	DELETE	/api/accounts/:id	Delete DEPARTMENT/C AMPAIGN unit	departments.edit
Sites	GET	/api/sites	List office location sites (HQ, Candelaria, etc.)	Authenticated
Audit Logs	GET	/api/audit-logs	Query system audit trail	auditlogs.view

Audit Logs	POST	/api/audit-logs/:id/undo	Undo an audited action	auditlogs.undo
Notifs	GET	/api/notifications	List user in-app notifications	Authenticated
Notifs	POST	/api/notifications/read-all	Mark all notifications as read	Authenticated
Users	GET	/api/users	List system user accounts	users.manage
Users	PUT	/api/users/:id/approve	Approve pending user registration	users.manage
Users	PUT	/api/users/:id/capabilities	Update individual capability overrides	users.manage
Roles	GET	/api/roles	List all system roles	users.manage
Roles	PUT	/api/roles/:slug	Update role capability matrix	roles.manage
Settings	GET	/api/settings	Retrieve global application settings	settings.manage

Detailed Request / Response Payloads for Core Endpoints

1. Authentication Step 1 (POST /api/auth/login)

Request Body:

```
{
  "email": "admin@bigoutsource.com",
  "password": "SecurePassword123!"
}
```

Success Response (200 OK):

```
{
  "success": true,
  "message": "Verification code sent to your email",
  "mfaRequired": true,
  "mfaToken": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9..."
}
```

Error Response (401 Unauthorized):

```
{
  "success": false,
  "message": "Invalid email or password"
}
```

2. Authentication Step 2 MFA Verification (POST /api/auth/login/mfa)

Request Body:

```
{
  "mfaToken": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
  "code": "482915"
}
```

Success Response (200 OK):

```
{
  "success": true,
  "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.full_session_jwt...",
  "user": {
    "id": "c7b3a1a9-9e8d-4f12-a345-6789abcdef01",
    "email": "admin@bigoutsource.com",
    "fullName": "System Admin",
    "role": "super_admin",
  }
}
```

```
    "capabilities": ["employees.view", "employees.create", "users.manage"]
  }
}
```

3. Create Employee (POST /api/employees)

Request Body:

```
{
  "name": "Juan Dela Cruz",
  "account": "Customer Support Campaign",
  "site": "HQ",
  "phoneNumber": "+639171234567",
  "bigoutsourceEmail": "j.delacruz@bigoutsource.ph",
  "employeeStatus": "Probationary",
  "dateHired": "2026-08-01",
  "position": "Support Specialist"
}
```

Success Response (201 Created):

```
{
  "success": true,
  "data": {
    "id": "e89f2a41-3b7c-4d1e-a902-123456789abc",
    "name": "Juan Dela Cruz",
    "account": "Customer Support Campaign",
    "site": "HQ",
    "bigoutsourceEmail": "j.delacruz@bigoutsource.ph",
    "lmsAccount": "j.delacruz",
    "status": "active",
    "employeeStatus": "Probationary",
    "provisioningStatus": "pending_it",
    "createdAt": "2026-08-11T08:30:00.000Z"
  }
}
```

4. Update Employee IT Details (PUT /api/employees/:id)

Request Body:

```
{
  "pcName": "PC-SUP-042",
  "rustdeskId": "987 654 321",
  "esetStatus": "active",
  "activityWatchStatus": "active",
}
```

```
{
  "windowsLicenseKey": "XXXXX-XXXXX-XXXXX-XXXXX-XXXXX"
}
```

Success Response (200 OK):

```
{
  "success": true,
  "data": {
    "id": "e89f2a41-3b7c-4d1e-a902-123456789abc",
    "pcName": "PC-SUP-042",
    "rustdeskId": "987 654 321",
    "eset": "active",
    "activitywatch": "active",
    "windowsLicenseKey": "XXXXX-XXXXX-XXXXX-XXXXX-XXXXX"
  }
}
```

5. Deployment & Operations

Complete Environment Variables Matrix

Variable Name	Purpose	Required / Optional	Dev Default	Production Example
NODE_ENV	Runtime environment mode	Required	development	production
PORT	Node API HTTP port	Required	5001	5001

DATABASE_URL	PostgreSQL connection string	Required	postgresql://postgres:postgres@127.0.0.1:5433/eims?schema=public	postgresql://user:pass@db:5432/eims
JWT_SECRET	Secret key for signing session tokens	Required	super_secret_jwt_key_for_local_dev	LONG_RANDOM_256BIT_SECRET_STRING
CORS_ORIGIN	Allowed cross-origin domains	Required	http://localhost:3000,http://localhost:3001	https://eims.bigoutsource.ph
SEED_SUPER_ADMIN_EMAIL	Admin account email bootstrapped on start	Optional	kamote@gmail.com	admin@bigoutsource.ph
SEED_SUPER_ADMIN_PASSWORD	Admin initial password	Optional	kamote123	ComplexInitialPass123!
SMTP_HOST	Outbound SMTP email host	Optional (Dev)	localhost	mail.bigoutsource.ph
SMTP_PORT	Outbound SMTP port	Optional (Dev)	1025	587

SMTP_USER	Outbound SMTP authentication username	Optional	<i>(empty)</i>	eims@bigoutsource.ph
SMTP_PASS	Outbound SMTP authentication password	Optional	<i>(empty)</i>	SMTP_Password_123
VITE_API_BASE_URL	Frontend API base endpoint	Required (Frontend)	http://localhost:5001/api	https://eims.bigoutsource.ph/api

Build & Deployment Workflow (Docker & Nginx)

For self-hosted production deployment (e.g. AWS EC2, DigitalOcean, or private company server), create a docker-compose.prod.yml file:

```

services:
  backend:
    build:
      context: ./backend
      dockerfile: Dockerfile
    command: sh -c "npx prisma generate && npx prisma db push && npm start"
    ports:
      - "5001:5001"
    env_file:
      - ./backend/.env
    environment:
      - NODE_ENV=production
      - DATABASE_URL=postgresql://postgres:postgres_prod_password@db:5432/eims
    depends_on:
      - db
    restart: unless-stopped

  frontend:
    build:

```

```

context: ./frontend
dockerfile: Dockerfile
args:
  - VITE_API_BASE_URL=https://eims.bigoutsource.ph/api
ports:
  - "3000:80"
depends_on:
  - backend
restart: unless-stopped

db:
  image: postgres:15-alpine
  restart: always
  environment:
    - POSTGRES_USER=postgres
    - POSTGRES_PASSWORD=postgres_prod_password
    - POSTGRES_DB=eims
  volumes:
    - pgdata_prod:/var/lib/postgresql/data

volumes:
  pgdata_prod:

```

Launch Command:

```
docker compose -f docker-compose.prod.yml up -d --build
```

Reverse Proxy & SSL Setup (Nginx Proxy Manager):

1. Point domain DNS records to your server IP:
 - eims.bigoutsource.ph -> Frontend (port 3000)
 - api.bigoutsource.ph -> Backend (port 5001)
2. Use Nginx Proxy Manager or Certbot to issue free SSL certificates via Let's Encrypt for both hostnames.

Backup Routines, Logging & Monitoring

- Database Backup Routine:
Schedule a daily cron job on the host server to dump the PostgreSQL database:

```
0 2 * * * docker exec -t $(docker ps -qf "name=db") pg_dump -U postgres eims | gzip > /backups/eims_db_$(date +%Y%m%d).sql.gz
```
- File Uploads Backup:
Periodically rsync or copy the backend/uploads directory to secure remote storage.

- **Application Health Checks:**
Monitor server health via endpoint GET `http://localhost:5001/health`. Returns `{ "success": true, "message": "API is healthy" }`.
- **System Audit Logs:**
All administrative actions (modifying employee records, changing roles, archiving employees) are logged to the AuditLog table and viewable under `/logs` by users with `auditlogs.view` permission.

6. Non-Technical Admin Guide

Welcome to the Operational Guide for HR Managers, IT Personnel, and Department Heads. This section explains how to perform daily administrative operations using the EIMS web interface.

Managing User Accounts & Access

Inviting & Approving New System Users

1. Log in as an Administrator and navigate to User Management (`/users`).
2. Click **Register Account**.
3. Fill out the user's Full Name, Email, Department, and assigned Site.
4. Click **Submit**. The system automatically sends an email invitation link to the user.
5. The employee opens the email link to set their secret password and verify their account using a 6-digit email code.
6. If manual approval is enabled, navigate to the Pending Users tab in User Management and click **Approve Account**.

Managing EMPLOYEE INFORMATION Records

Adding a New Employee

1. Navigate to Directory (`/directory`) and click **+ Add Employee**.
2. Complete the EMPLOYEE INFORMATION form:
 - **HR Details:** Full Name, Phone Number, Date Hired, Employment Status (Regular/Probationary), Birthdate, Personal Email, Emergency Contacts, SSS, TIN, PhilHealth, and Pag-IBIG numbers.
 - **DEPARTMENT/CAMPAIGN:** Select the campaign or department unit where the employee belongs.

- Site Location: Choose HQ, Candelaria, WFH, or Hybrid.
3. Save the record. An automated notification will alert IT that new IT provisioning is required.

Updating IT & Workstation Information

1. Open the employee's profile by clicking their name in the Directory.
2. Under the IT Credentials & Hardware section, update:
 - Corporate email & LMS account details.
 - PC Name and workstation hardware details.
 - REMOTE ID (Remote connection ID for IT support).
 - ESET Antivirus status (active / inactive) and ActivityWatch status (active / inactive).
 - Disk Encryption Key and Windows License Key.
3. Click Save Changes.

Employee Archival (Deactivation) & Reactivation Workflow

To maintain accurate records when an employee leaves or shifts status, EIMS enforces a clearance workflow:

Active Employee —▶ Deactivation Initiated —▶ IT Clearance —▶ HR Final Archival —▶ Archived

1. Initiating Archival:
 - Navigate to the employee's profile and click Deactivate & Send Archive Request.
 - Fill in the separation date and separation reason.
2. IT Clearance:
 - IT Admins receive an alert to clear accounts, collect company hardware, and revoke access keys.
 - IT updates the employee's status to Ready for Archive.
3. HR Final Archival:
 - HR receives a notification that IT clearance is complete.
 - HR clicks Finalize Archival to move the employee to the Archived Employees tab.
4. Reactivating an Employee:
 - Go to Directory -> Archived Employees.
 - Click Unarchive. The employee is reactivated and IT is notified to reprovision credentials.

Managing DEPARTMENT/CAMPAIGN Units

1. Navigate to Departments (/departments).
2. Click + Add Department/Campaign.
3. Enter the Department Code (e.g. CS-01) and Department/Campaign Name (e.g. Customer Support Campaign).
4. Click Save. The new DEPARTMENT/CAMPAIGN unit is immediately available across all employee creation forms, search filters, and analytics reports.

Bulk Importing Employees via Excel / CSV

1. Navigate to Employee Imports (/employee-imports/issues).
2. Upload your company spreadsheet (.xlsx or .csv).
3. The system parses all rows and presents a Staging Review Table:
 - Green Rows: Ready for instant import.
 - Yellow/Red Rows: Duplicate names or invalid email formats detected.
4. Resolve duplicate conflicts by choosing to Merge, Overwrite, or Skip.
5. Click Import Ready Rows. All valid entries are added to the live employee directory in bulk.